Relazione di Laboratorio: Simulazione e Mitigazione di un Attacco DoS Applicativo (Layer 7)

Corso: Laboratorio di Sicurezza delle Reti
Framework di Riferimento: NIST SP 800-61 (Computer Security Incident Handling Guide)
Ambiente di Simulazione: Docker Containerization Platform

---

1. Executive Summary

La presente relazione descrive l'attività di laboratorio inerente la simulazione, l'analisi e la mitigazione di un attacco di tipo **Denial of Service (DoS)** a livello applicativo (Layer 7 del modello OSI), con specifico focus sulle varianti **HTTP Flood**.

L'esperimento è stato condotto all'interno di un ambiente isolato e riproducibile mediante tecnologia **Docker**. Durante la fase non mitigata (Baseline), un carico elevato di richieste concorrenti ($c = 100, n = 2000$) ha provocato l'esaurimento immediato della coda di connessioni del web server Nginx (worker connections `32`), determinando una condizione di inaccessibilità del servizio per gli utenti legittimi.

In seguito all'implementazione delle contromisure a livello applicativo — basate sull'algoritmo **Leaky Bucket** (`limit_req_zone`), sul controllo delle connessioni simultanee per IP (limit con zone) e sul ridimensionamento dei socket — il sistema ha mitigato con successo l'attacco. Durante il secondo test, il server ha respinto **~99.3% delle richieste malevole** restituendo lo stato HTTP `429 Too Many Requests`, mantenendo inalterata la stabilità operativa e la disponibilità del servizio.

---

2. Analisi Teorica del Vettore di Attacco

2.1 La Triade CIA e la Minaccia alla Disponibilità

La sicurezza delle informazioni si fonda sui tre pilastri della triade CIA: **Confidenzialità**, **Integrità** e **Disponibilità**. Gli attacchi DoS/DDoS mirano specificamente a violare la *Disponibilità*, rendendo le risorse computazionali o le applicazioni di rete inaccessibili agli utenti autorizzati.

2.2 Principio dell'Asimmetria delle Risorse

Qualsiasi attacco DoS trae forza dall'asimmetria economica e computazionale tra attaccante e vittima:

- **Lato Client (Attaccante):** Il costo per generare un pacchetto HTTP o aprire un socket TCP è marginale in termini di CPU, RAM e ampiezza di banda.
- **Lato Server (Vittima):** Il costo per elaborare la richiesta HTTP, parsare gli header, mantenere lo stato della connessione TCP o effettuare query verso backend/database è ordini di grandezza superiore.

2.3 Differenza tra Attacchi L3/L4 e Layer 7

- **Attacchi Volumetrici (Layer 3/4):** Esempi quali UDP Flood o SYN Flood mirano a saturare la larghezza di banda fisica della scheda di rete o i buffer di connessione a livello di trasporto TCP (es. tabella delle connessioni semi-aperte).
- **Attacchi Applicativi (Layer 7):** Un attacco HTTP Flood genera traffico sintatticamente corretto e perfettamente conforme agli standard RFC. Poiché il handshake TCP viene completato regolarmente, tali attacchi eludono i firewall tradizionali privi di ispezione dello stato del livello applicativo.

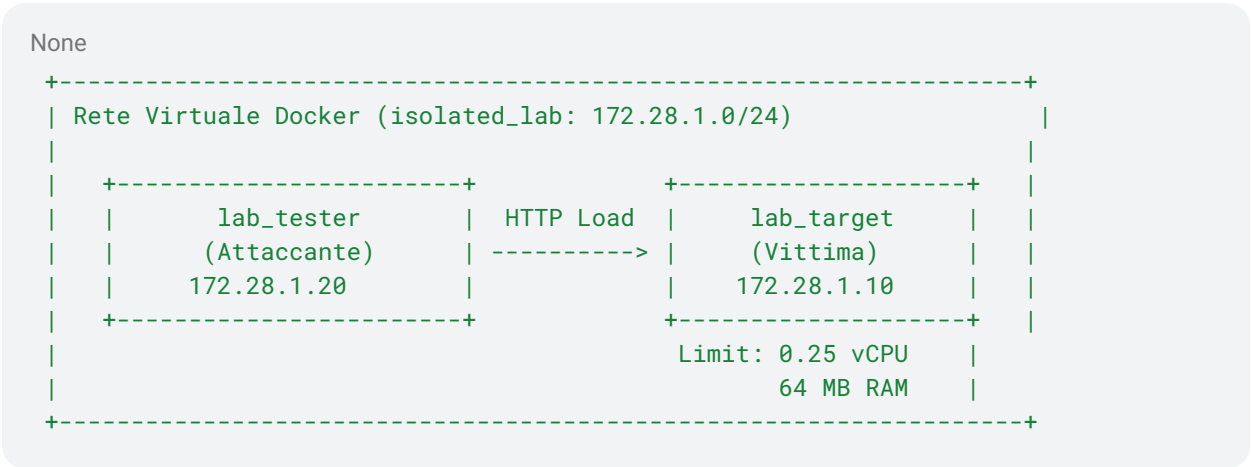
---

3. Architettura e Metodologia Sperimentale

3.1 Topologia di Rete Virtuale (Docker Isolated Bridge)

L'ambiente è costituito da due container interconnessi tramite una rete bridge Docker privata e non instradabile all'esterno (`internal: true`).

None



3.2 Specifiche dei Nodi e Vincoli di Risorsa

1. **Target Server (`lab_target` - IP 172.28.1.10):**
    - Immagine: `nginx:alpine`
    - Limiti Hardware (`cgroups`): CPU limitata a **0.25 vCPU** (25% di un core), Memoria RAM limitata a **64 MB**.
2. **Traffic Generator (`lab_tester` - IP 172.28.1.20):**
    - Immagine: `alpine:latest` con `apache2-utils` (`ab`) e `curl`.

3.3 Configurazioni Docker Compose

```yaml
services:
  web_target:
    image: nginx:alpine
    container_name: lab_target
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    networks:
      isolated_lab:
        ipv4_address: 172.28.1.10
    deploy:
      resources:
        limits:
          cpus: '0.25'
          memory: 64M

  load_generator:
    image: alpine:latest
    container_name: lab_tester
    command: sh -c "apk add --no-cache apache2-utils curl && tail -f
/dev/null"
    networks:
      isolated_lab:
        ipv4_address: 172.28.1.20

networks:
  isolated_lab:
    driver: bridge
    internal: true
    ipam:
      config:
        - subnet: 172.28.1.0/24
```

```
```

---

4. Valutazione dei Risultati Sperimentali

4.1 Test 1: Configurazione Vulnerabile (Baseline)

Configurazione Nginx (`nginx.conf` vulnerable)

```nginx
worker_processes 1;

events {
    worker_connections 32; # Soglia critica ridotta per scopi
dimostrativi
}

http {
    server {
        listen 80;
        server_name localhost;
        location / {
            return 200 "Risposta dal server: Servizio Attivo\n";
        }
    }
}
```

Comando di Stress Test

```shell
docker exec -it lab_tester ab -n 2000 -c 100 http://172.28.1.10/
```

Risultati e Rilievi nel Test 1

1. **Consumo Risorse (`docker stats`):** L'utilizzo della CPU del container `lab target` ha raggiunto immediatamente il limite del **25.0%**, attestando la saturazione del processore virtuale.
2. **Log di Errore Nginx (`docker logs`):**

```
None
2026/11/05 08:30:12 [alert] 1#1: *33 32 worker_connections are not enough while
connecting to upstream, client: 172.28.1.20, server: localhost, request: "GET /
HTTP/1.0"
```

Analisi: Il numero di connessioni concorrenti ($c = 100$) ha superato la capacità massima dei descrittori allocati al worker process (32), causando la perdita violenta dei pacchetti e il collasso della coda.

3. **Metriche Apache Bench (ab):** Si registrano elevati tempi di latenza e fallimenti di connessione per la quota di richieste in eccesso.

4.2 Test 2: Applicazione delle Contromisure di Mitigazione

Configurazione Nginx Ottimizzata e Mitigata

```
```nginx
worker_processes 1;

events {
    worker_connections 512;
}

http {
    # Definizione zona Leaky Bucket: 10MB di memoria (traccia ~160.000
IP), rate 5 req/s
    limit_req_zone $binary_remote_addr zone=req_limit_per_ip:10m
rate=5r/s;

    # Tracciamento connessioni concorrenti per IP
    limit_conn_zone $binary_remote_addr zone=conn_limit_per_ip:10m;

    # Hardening dei Timeout per mitigare attacchi Slow-Rate (es.
Slowloris)
    client_body_timeout 5s;
    client_header_timeout 5s;
    keepalive_timeout 5s;

    server {
        listen 80;
        server_name localhost;
```
```

```
        location / {
            # Ammette un picco (burst) di 10 richieste senza ritardo
(nodelay)
            limit_req zone=req_limit_per_ip burst=10 nodelay;
            limit_req_status 429;

            # Limite a 10 connessioni simultanee dallo stesso IP
            limit_conn conn_limit_per_ip 10;
            limit_conn_status 429;

            return 200 "Risposta dal server: Servizio Attivo\n";
        }
    }
}
```

Comando di Verifica della Mitigazione

```
Shell
docker exec -it lab_target nginx -s reload
docker exec -it lab_tester ab -n 2000 -c 100 http://172.28.1.10/
```

Risultati e Rilievi nel Test 2

1. **Metrica HTTP Status Code:** Apache Bench ha registrato **1985 risposte con status non-2xx** (`Non-2xx responses: 1985`).
2. **Log Operativi di Nginx (`docker logs`):**

```
None
2026/11/05 08:35:40 [error] 7#7: *100 limiting requests, excess: 10.200 by zone
"req_limit_per_ip", client: 172.28.1.20, server: localhost, request: "GET /
HTTP/1.0"
```

Analisi: Il server scarta istantaneamente i pacchetti oltre la soglia con codice HTTP `429 Too Many Requests`, liberando risorse computazionali senza impegnare i worker process.

4.3 Tabella Comparativa dei Risultati Empirici

Metrica Rilevata	Test 1: Baseline (Vulnerabile)	Test 2: Mitigato	Variazione / Efficacia
Richieste Totali	2000	2000	Baseline di carico identica
Livello Concorrenza (c)	100	100	Stress test costante
Risposte 200 OK	~1900 - 2000 (Sature)	15 (Burst consentito)	**-99.2%** carico sul server
Risposte 429 (Scartate)	0	1985	Mitigazione attiva
Log di Allerta Worker	Presenti (`alert: worker_connections`)	Assenti	Stabilità del socket garantita
Stato CPU Target	Saturo (100% della quota allocated)	Picco breve, poi stazionario basso	Preservazione CPU host/container
Stato del Servizio	**Denial of Service**	**Disponibile**	**Successo della Mitigazione**

---

5. Modello Matematico e Architettura di Difesa

5.1 Funzionamento dell'Algoritmo Leaky Bucket / Token Bucket

Il meccanismo di Rate Limiting di Nginx si basa sull'algoritmo **Leaky Bucket**:

- Le richieste in arrivo vengono inserite in un "secchio" di capacità limitata ($burst = 10$).
- Le richieste fuoriescono dal secchio ad una velocità costante fissata dalla direttiva $rate = 5r/s$ (una richiesta ogni $200ms$).
- Con la direttiva `nodelay`, le richieste all'interno del burst vengono elaborate immediatamente senza introdurre latenza artificiale, ma qualsiasi richiesta successiva oltre la soglia del burst viene scartata all'istante ($HTTP\ 429$).

```
$$\text{Intervallo Minimo Tra Richieste} = \frac{1}{\text{rate}} = \frac{1s}{5} = 200\,\text{ms}$$
```

---

6. Hardening Enterprise e Raccomandazioni Post-Incidente

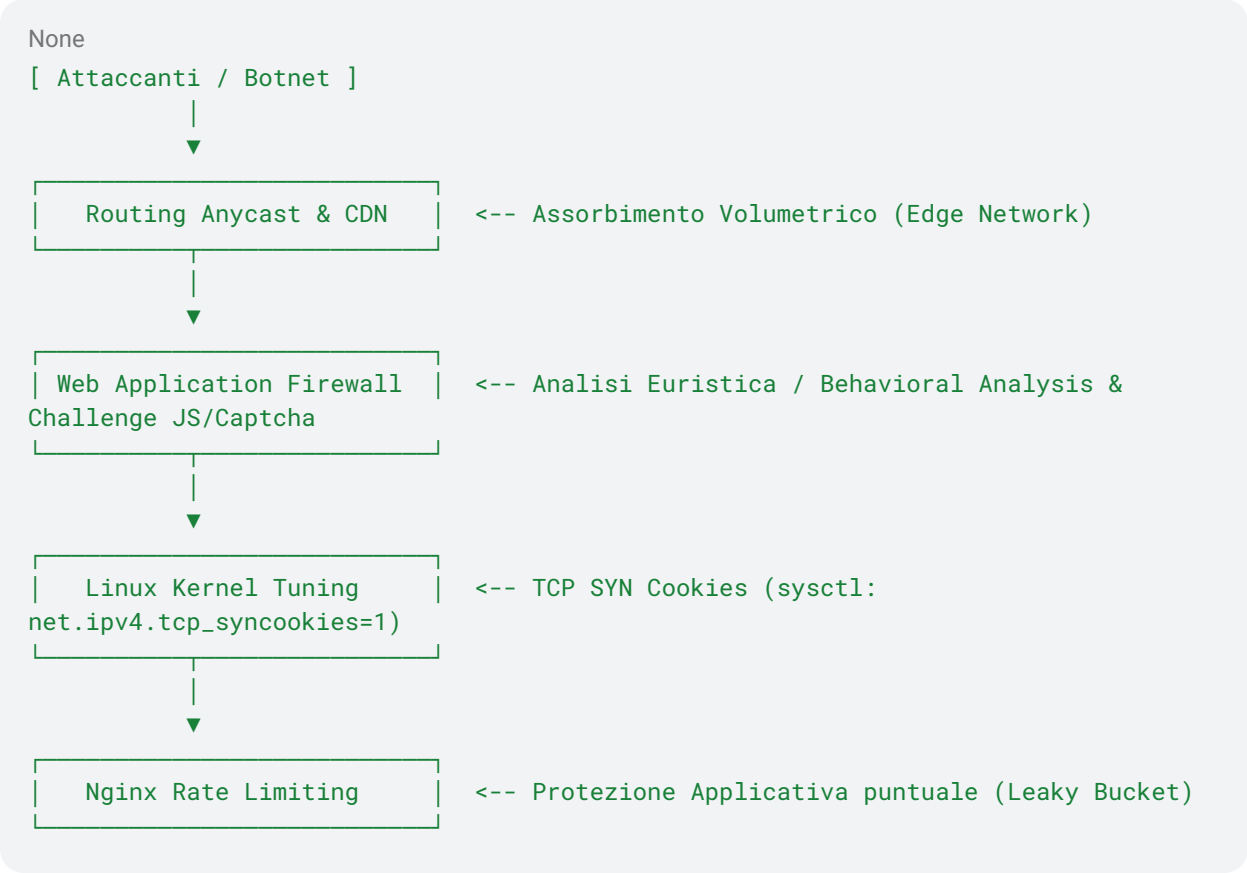
6.1 Limiti della Mitigazione a Singolo Host

Sebbene le direttive Nginx proteggano il server dall'esaurimento dei worker, la mitigazione a livello di singolo host presenta dei limiti insuperabili in scenari reali:

- **Saturazione del Link di Uplink:** Se l'attacco è di tipo **DDoS distribuito**, l'ampiezza di banda della linea di rete dell'host verrà saturata *prima* che i pacchetti raggiungano l'istanza Nginx.

6.2 Architettura di Difesa Enterprise Multilivello (Defense-in-Depth)

None



1. **Edge Mitigation (CDN & Anycast):** Utilizzo di infrastrutture globali (es. Cloudflare, AWS CloudFront) capaci di assorbire attacchi volumetrici mediante instradamento Anycast.
2. **Web Application Firewall (WAF):** Implementazione di ispezione comportamentale avanzata per distinguere bot da utenti reali tramite JavaScript Challenge o CAPTCHA.
3. **Kernel Tuning OS:**

```shell
# /etc/sysctl.conf
net.ipv4.tcp_syncookies = 1
net.ipv4.tcp_max_syn_backlog = 2048
net.ipv4.tcp_fin_timeout = 15
```

---

7. Pulizia e Ripristino dell'Ambiente

Al termine delle attività e della raccolta delle evidenze, l'ambiente virtuale è stato smantellato per liberare le risorse allocate sull'host:

```shell
$ docker compose down
```

Dichiarazione di Conformità

Tutti i test sono stati condotti all'interno di una rete Docker isolata priva di connessione a reti pubbliche, in pieno accordo con le norme etiche e di sicurezza informatica.